Quiz 2: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 2 is Thursday Sep 10, at the end of class. ~12 minutes, 25 points.

Covers: the Week 2 lecture (Sep 3) and **Lab 2**.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed.

A formula sheet is handed out with the quiz (`formula-sheet.md`), so nothing needs memorizing. Read it now rather than on Thursday: it lists **more formulas than any one question needs**, so deciding which applies is still your job.

The goal is not memorization. It is showing that you can take an architecture and work out its shapes, its parameter count, and its forward pass by hand, and that you can say why a nonlinearity is not optional.

How to use this guide

The quiz has the same shape as Quiz 1. Roughly 10 points are one small network worked end to end: shapes, parameter count, forward pass, one layer computed by hand. Roughly 6 points are short conceptual answers. The remaining 9 points ask you to **explain what Lab 2 demonstrated**, which is why running it yourself matters.

No question asks you to recall a number from your lab. The lab questions state what happened and ask *why*, so the way to prepare is to scroll back through your own output until you can account for each result.

If you can do everything in Parts 1 through 3 below on paper without notes, and you can explain the seven observations in Part 7, you are ready.

Part 1: A layer is a matrix multiply

One neuron takes an input vector, weighs each component, adds a bias, and passes the total through a function:

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad a = \sigma(z)$$

Put several neurons side by side **in the same layer**. They all see the same $\mathbf{x}$, and each has its own weight vector and bias. Stack those weight vectors as the **rows** of one matrix and the whole layer is one line:

$$\boxed{\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}}$$

One row of W per unit. This is the single most useful fact for the shape questions.

Shapes, and the two rules that resolve everything

Object	Shape
$\mathbf{W}$ for a layer	$(n_{\text{out}} \times n_{\text{in}})$
$\mathbf{b}$ for a layer	(n_{out})
batch input $\mathbf{X}$	$(N \times n_{\text{in}})$, one row per example

1. **Columns of W must match rows of x**, or the multiply is undefined.
2. **The bias has one entry per unit**, never per input.

Be able to write down every $\mathbf{W}$ and $\mathbf{b}$ for an architecture given only as something like $3 \to 4 \to 3$.

Parameter counting

Per layer:

$$n_{\text{out}} \times n_{\text{in}} + n_{\text{out}}$$

Do it layer by layer and add. **Forgetting the biases is the most common error on this question.** For the lecture's 2-3-2-1 network:

$$\underbrace{(3 \times 2 + 3)}_{9} + \underbrace{(2 \times 3 + 2)}_{8} + \underbrace{(1 \times 2 + 1)}_{3} = 20$$

Part 2: The forward pass

Feed an input in at the left, apply each layer in turn, read the prediction off the right:

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{a}^{(\ell)} = \sigma^{(\ell)}\left(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \qquad \hat{\mathbf{y}} = \mathbf{a}^{(L)}$$

- The superscript is a **layer index, not a power**.
- $\mathbf{a}^{(0)} = \mathbf{x}$ is bookkeeping, so that one formula covers every layer including the first.
- **This is everything a trained model does at prediction time.** Training is only the process of choosing the numbers in the $\mathbf{W}$s and $\mathbf{b}$s.

Be able to compute one layer **by hand**: a 2×2 matrix times a 2-vector, plus a bias, then an activation applied **elementwise**. Watch ReLU on a negative pre-activation: the answer is exactly $\mathbf{0}$, not the negative number and not its absolute value.

Batches

The batch dimension comes **first** and the layers never touch it. Rows do not interact inside a layer, so pushing 64 examples through at once gives exactly what pushing one through 64 times gives. That is why the same code handles both.

Part 3: Activations

Hidden layers

Activation	Range	Note
sigmoid	$(0, 1)$	saturates; flat slope far from 0 means vanishing gradients
tanh	$(-1, 1)$	same saturation problem, but zero-centered
ReLU	$[0, \infty)$	slope 1 for positive z, cheap, sparse. **The default today.**

The output layer is not a free choice

It is dictated by the problem:

Problem	Output activation	Loss
Scalar regression	none (identity)	MSE or MAE
Binary classification	sigmoid	binary cross-entropy
Multi-class, one label	softmax	categorical cross-entropy

Know **why** softmax rather than per-unit sigmoid for multi-class: the class probabilities have to sum to 1, and sigmoid applied to each unit separately does not guarantee that.

Why a nonlinearity is not optional

This is the load-bearing idea of the whole week. Stack linear layers with no activation and they **collapse into a single linear layer**:

$$\mathbf{W}^{(3)}\left(\mathbf{W}^{(2)}\left(\mathbf{W}^{(1)}\mathbf{x}\right)\right) = \left(\mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)}\right)\mathbf{x} = \mathbf{W}_{\text{eff}}\,\mathbf{x}$$

A composition of linear maps is linear. The depth buys **no expressive power at all**: your 10-layer network can represent nothing that linear regression cannot. Inserting any nonlinearity breaks the collapse, because it cannot be absorbed into a matrix product.

Be able to state this in one sentence, with the reason.

Part 4: Depth, width, and universal approximation

The **universal approximation theorem** says a network with a single hidden layer, given enough units, can approximate essentially any continuous function to any accuracy you like.

What it does not promise, which is the examinable part:

- Not that the network is **trainable**. Good weights existing is not the same as gradient descent finding them.
- Not **how many units** you need, which can be astronomically large.
- Not that it will **generalize** to data it has not seen.
- Not that shallow is **efficient**. Depth often reaches the same accuracy with far fewer parameters, which is why we go deep in practice.

The useful conclusion: **the model family is not your limitation.** Optimization and data are.

Part 5: Vocabulary

You need these cold, and one small calculation:

Term	Meaning
epoch	one full pass over the training set
iteration	one parameter update, using one batch
batch size	how many examples per update
learning rate λ	how far to step along the gradient

$$\text{iterations per epoch} = \frac{N_{\text{train}}}{\text{batch size}}$$

So 640 training examples at batch size 64 is 10 iterations per epoch.

Part 6: Why gradients are the next problem

Gradient descent needs $\partial\mathcal{L}/\partial w$ for **every** parameter. You can approximate each one with a finite difference:

$$\frac{\partial \mathcal{L}}{\partial w_j} \approx \frac{\mathcal{L}(\mathbf{w} + h\mathbf{e}_j) - \mathcal{L}(\mathbf{w})}{h}$$

The cost is the point: **one forward pass per parameter, plus one** for the baseline loss. That is $P + 1$ passes for a single gradient step, and training needs thousands of steps. It is fine at 49 parameters and hopeless at 25 million. Next week replaces it with backpropagation, which computes the same gradient in roughly the cost of one forward and one backward pass, regardless of how many parameters there are.

Part 7: From Lab 2

9 of the 25 points come from Lab 2, and none of them ask for a number. Everyone ran the same code, so the questions state what happened and ask you to explain the mechanism. Open your notebook, look at your own output, and make sure you can say each of these in a sentence or two:

1. **Why a layer is a matrix multiply**, and why **W** is shaped `(units, inputs)` rather than the other way around.
2. **Why the batched forward pass needs a transpose** relative to the single-example version, and why the two still agree.
3. **Why a 2-3-2-1 network has exactly 20 parameters.** The pattern per layer.
4. **Why stacked linear layers collapse.** What the extra 95 parameters bought you in Part 3, and what inserting tanh changed.
5. **Why finite-difference gradients cost one extra forward pass per parameter**, plus one for the unnudged loss.

6. **Why that cost is fatal** once the model has millions of parameters, even though the method produced a perfectly good fit on 49 parameters.
7. **What next week is replacing.** Autograd computes the same gradient; it does not take 25 million extra forward passes to do it.

These are the same seven points listed at the end of the lab notebook.

Checklist

Before Thursday, confirm you can:

- ☐ Write every $\mathbf{W}$ and $\mathbf{b}$ shape for an architecture given as $3 \to 4 \to 3$
- ☐ Count the parameters layer by layer, biases included
- ☐ Write the forward pass with the right activation on each layer
- ☐ Compute one layer by hand, applying ReLU elementwise and clipping negatives to 0
- ☐ Give the batch input and output shapes, and iterations per epoch
- ☐ State what stacking linear layers with no activation collapses to, and why
- ☐ Name the output activation and loss for regression, binary, and multi-class
- ☐ Say why softmax rather than per-unit sigmoid for multi-class
- ☐ Name two things universal approximation does not promise
- ☐ Define epoch, iteration, batch size, and learning rate
- ☐ Explain where the $P + 1$ forward passes come from, and why that is fatal at scale
- ☐ Explain all seven Lab 2 observations in Part 7, from your own output
- ☐ Arrive with a calculator and nothing else; no notes are permitted